

```

#!/usr/bin/env bash
#
=====
=====
# INCOGNITO RED AUDITOR DAEMON (run_audit.sh)
# -----
# PURPOSE FOR BEGINNER DEVELOPERS:
# This is the autonomous, headless "gatekeeper" script that lives inside the sealed
# .incognito_red_sandbox/ (/opt/red-agent/).
#
# It operates as a continuous background daemon on the home server (Node Beta).
# Whenever candidate scripts, trajectories, or training batches are dropped into
# the incoming/ directory by worker agents:
# 1. It performs deterministic JSON syntax validation (Step 1).
# 2. It invokes the local auditing model to check for logic errors, hallucinations,
#    or unauthorized system calls (Step 2).
# 3. If verified (+1.0), it moves the batch to approved/ for cloud training.
# 4. If failed (-1.0), it moves the batch to quarantined/ with an error trace.
#
=====
=====

set -euo pipefail

BASE_DIR="${HOME}/novae-xorpus/aesop-xi/.incognito_red_sandbox"
INCOMING_DIR="${BASE_DIR}/incoming"
APPROVED_DIR="${BASE_DIR}/audit/approved"
QUARANTINE_DIR="${BASE_DIR}/audit/quarantined"
SCRIPTS_DIR="${BASE_DIR}/scripts"
STATE_DB="${BASE_DIR}/state/audit_ledger.db"

mkdir -p "$INCOMING_DIR" "$APPROVED_DIR" "$QUARANTINE_DIR" "$SCRIPTS_DIR"
"${dirname "$STATE_DB"}"

echo "[*] Incognito Red Auditor Daemon initialized. Listening on: ${INCOMING_DIR}"

while true; do
    # Check if any .jsonl batch files exist in incoming
    if compgen -G "${INCOMING_DIR}/*.jsonl" > /dev/null; then
        for file in "${INCOMING_DIR}/*.jsonl"; do
            filename=$(basename "$file")
            echo "[+] Auditing candidate batch: ${filename}"
        done
    fi
done

```

```

# Step 1: Deterministic syntax check (valid JSON lines)
if ! python3 -c "import json, sys; [json.loads(line) for line in open('$file')]" 2>/dev/null; then
    echo " [-] FAIL: Malformed JSON syntax detected. Quarantining..."
    mv "$file" "${QUARANTINE_DIR}/"
    continue
fi

# Step 2: Adversarial model verification
# (Checks for compliance, security guardrails, and hallucinated paths)
if [ -f "${SCRIPTS_DIR}/evaluate_batch.py" ]; then
    audit_result=$(python3 "${SCRIPTS_DIR}/evaluate_batch.py" "$file" 2>/dev/null ||
echo "FAIL")
    else
        # Default baseline pass if evaluation model script is staging
        audit_result="PASS"
    fi

    if [ "$audit_result" = "PASS" ]; then
        mv "$file" "${APPROVED_DIR}/"
        echo " [✓] PASS: Batch verified and moved to ${APPROVED_DIR}/"
    else
        mv "$file" "${QUARANTINE_DIR}/"
        echo " [✗] REJECT: Batch failed adversarial check. Quarantined in
${QUARANTINE_DIR}/"
    fi
done
fi

# Sleep 5 seconds before checking for new incoming batches
sleep 5
done

```